Asynchronous Programming in Python

Presented by :Ahmed Hossam Rafik
Supervised by : Eng Mohamed El Adawy

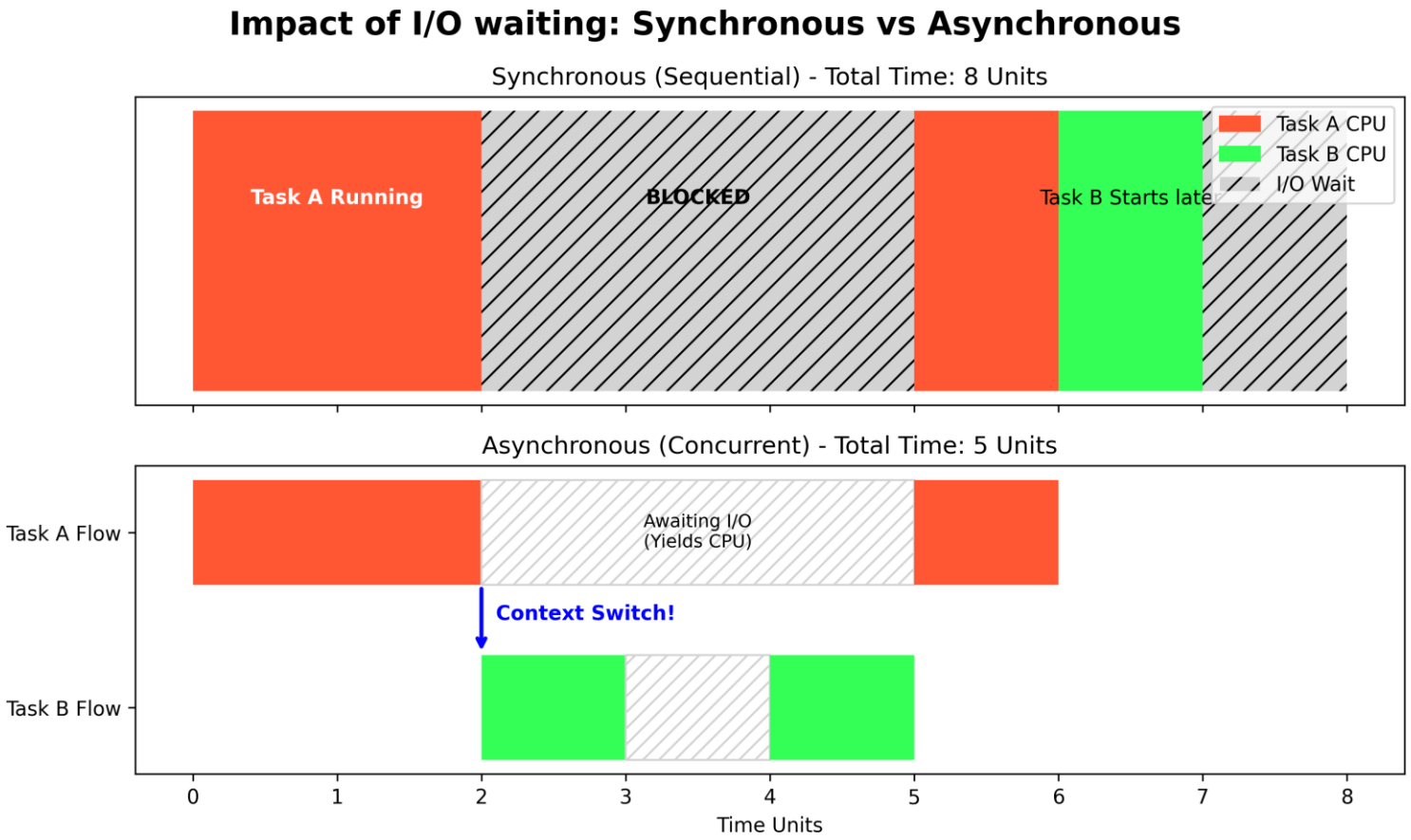
The Core Concept: Sync vs. Async

- Imagine programming is a journey from Point A to Point D. In traditional, synchronous programming, we travel in a straight line. If there is a delay at Point B, the entire journey stops. We just sit there waiting.

- Asynchronous programming changes the game. Instead of waiting, we send out scouts. We can start exploring paths B, C, and D even if the task at A isn't completely finished yet.

- By doing this, our program handles multiple tasks at the exact same time without sitting around doing nothing during idle waiting periods

The Core Concept: Sync vs. Async



The Benefit: Code handles multiple operations simultaneously, eliminating unnecessary waiting times.

Concurrency Models

Choosing the Right Concurrency Model:

- Async IO: Best for tasks with heavy waiting (Network requests, reading files). Very low CPU usage.

- Threads: Best for tasks that wait but also need to share memory/data. Runs in parallel within the same application.

- Processes: Best for CPU-heavy tasks (Intensive computations). Runs truly parallel across multiple CPU cores.

Event Loop

- The Heart of Async is the Event Loop as it acts as the central hub: Manages and distributes all tasks.

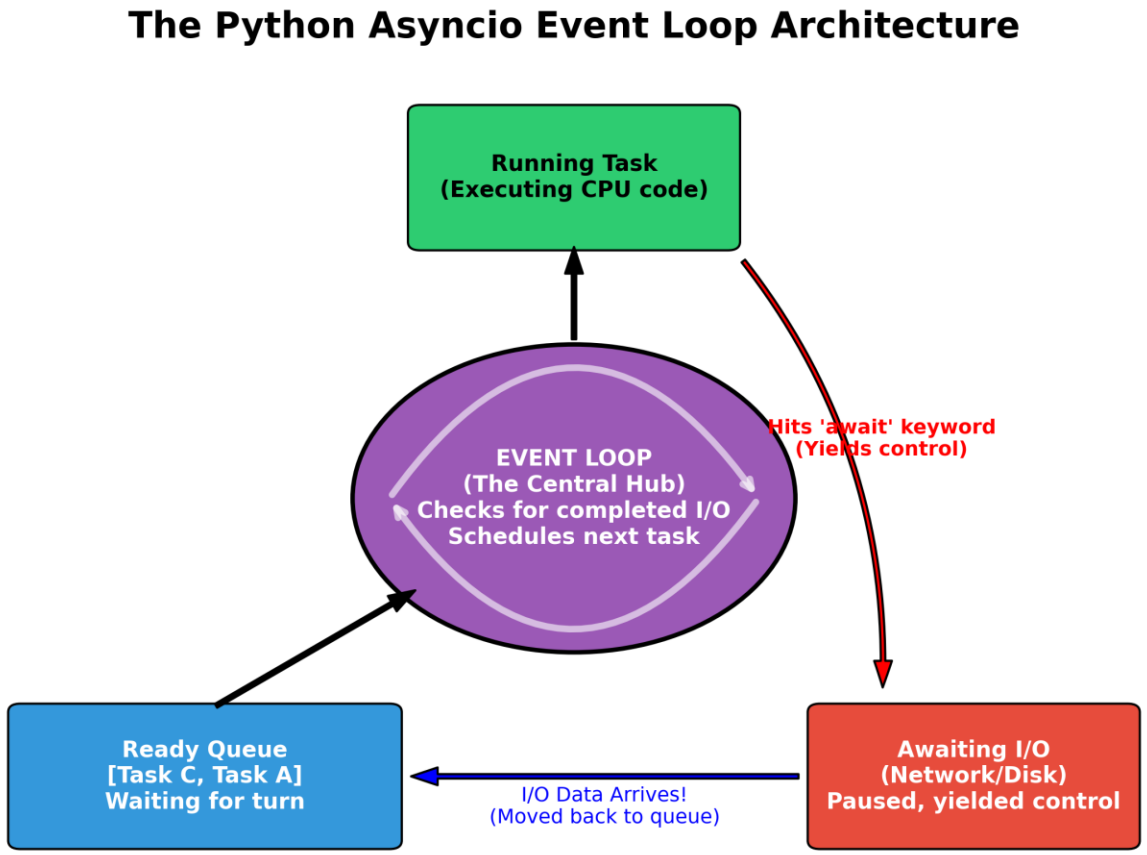
- How it Works:

❑Tasks circle the hub waiting for their turn.

❑Cooperative Yielding: Tasks step aside when waiting.

❑Resumption: Once the awaited operation finishes, the task resumes

Event loop



If you are using Async IO, the brain of the operation is the Event Loop.

Coroutines & The `await` Keyword

- What is a Coroutine? A function defined with 'async def’.
- Creation vs. Execution: Calling an async function only creates a coroutine object; it doesn't run it.
- The Trigger: The 'await' keyword is required to execute the coroutine and get its result.
- The Entry Point: 'asyncio.run()' starts the event loop and executes the main coroutine.

Coroutines & The `await` Keyword

```python
import asyncio

# couroutine function
async def main():
    print("Start of main coroutine")

main() -> Coroutine object

# Run the main coroutine
asyncio.run(main())
```

```
<coroutine object main at 0x1017a36c0>
```

So we need to pass coroutine object to async code run. It's going to wait for that to finish and it's going to start the event loop for us where it handles all of our asynchronous programing.

Managing Concurrency: Tasks

```python
# Define a coroutine that simulates a time-consuming task
async def fetch_data(delay, id):
    print("Fetching data... id:", id)
    await asyncio.sleep(delay)  # Simulate an I/O operation with a sleep
    print("Data fetched, id:", id)
    return {"data": "Some data", "id": id}  # Return some data


# Define another coroutine that calls the first coroutine
async def main():
    task1 = fetch_data(2, 1)
    task2 = fetch_data(2, 2)

    result1 = await task1
    print(f"Received result: {result1}")

    result2 = await task2
    print(f"Received result: {result2}")
```

```
Fetching data... id: 1
Data fetched, id: 1
Received result: {'data': 'Some data', 'id': 1}
Fetching data... id: 2
Data fetched, id: 2
Received result: {'data': 'Some data', 'id': 2}
```

It seems like they are not running concurrently and this is because code routine doesn't start running until it's awaited. So in this case we actually wait for the first code routine to finish and then start with the second and the solution here is create tasks

```python
task = fetch_data(2)
```

This will not be executed until the coroutine is awaited

```python
result = await task
print(f"Received result: {result}")
print("End of main coroutine")
```

Here we will not proceed the rest of the code until the task is finished

Managing Concurrency: Tasks

- asyncio.create_task(): Schedules a coroutine to run in the background immediately.
- asyncio.gather(): Runs multiple coroutines concurrently and collects their results into a list.
- TaskGroups: The modern way to group tasks (using 'async with TaskGroup') with built-in error handling.

Managing Concurrency: Tasks

```python
async def fetch_data(id, sleep_time):
    print(f"Coroutine {id} starting to fetch data.")
    await asyncio.sleep(sleep_time)
    return {"id": id, "data": f"Sample data from coroutine {id}"}


async def main():
    # Create tasks for running coroutines concurrently
    task1 = asyncio.create_task(fetch_data(1, 2))
    task2 = asyncio.create_task(fetch_data(2, 3))
    task3 = asyncio.create_task(fetch_data(3, 1))

    result1 = await task1
    result2 = await task2
    result3 = await task3

    print(result1, result2, result3)


asyncio.run(main())
```

They ran concurrently

```
Coroutine 1 starting to fetch data.
Coroutine 2 starting to fetch data.
Coroutine 3 starting to fetch data.
{'id': 1, 'data': 'Sample data from coroutine 1'} {'id': 2, 'data': 'Sample data from coroutine 2'} {'id': 3, 'data': 'Sample
 data from coroutine 3'}
```

Keeping it Safe: Synchronization Primitives

- Lock: Protects a shared resource. Ensures only one coroutine accesses it at a time.
- Semaphore: Limits access to a specific number of coroutines (e.g., max 5 network requests).
- Event: A simple Boolean flag that allows code to block until another part sets the flag to True.

Summary & Key Takeaways

- Efficiency Over Waiting: Async IO maximizes performance by managing many waiting tasks concurrently on a single thread.
- The Event Loop is the Engine: It cooperatively pauses and resumes tasks, ensuring the CPU is never idle during I/O delays.
- Coroutines & Tasks: Use 'async' and 'await' to define operations, and Tasks to run them simultaneously.
- Play it Safe: Always use Synchronization Primitives to protect shared data.

Thank You!